

Qt 開発への AI 適用とテスト

三点セットでテストを分類し、自動テストと
手動テストを分ける

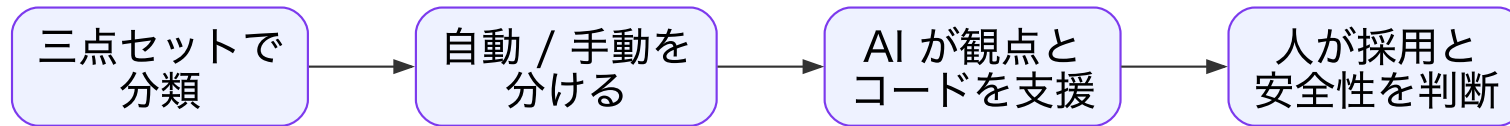
(株) SRA

Version 0.1.0

2026-05-28

1. 今日の結論

AI で生成したコードは、テストも分割して扱う。



テストは、AI 生成物を信頼するためではなく、仕様との差分を見つけるために使う。

2. テストの目的

AI は動くコードを作れるが、仕様どおりとは限らない。

見つけたいもの	例
未実装	仕様にある処理がコードにない
仕様外追加	AI が便利機能を足した
仮決め	未決事項を AI が決めた
境界値漏れ	0、空、上限、下限を扱っていない
例外漏れ	エラー条件、保存失敗、入力不正が抜ける

テストは、仕様と実装の差分を見つけるための仕組みである。

3. 三点セットがテスト分類の軸になる

三点セットは、テストケースを整理する棚になる。

仕様	テスト分類	確認すること
UC仕様書	シナリオテスト	利用者の目的、成功条件、例外条件
UI仕様書	UIテスト	画面、入力、表示、操作、エラー表示
ビジネスレイヤー仕様書	ロジックテスト	検証、計算、保存、状態変更

分類できると、テストケースがただの一覧ではなくなる。

漏れ、重複、偏りを見つけやすい。

4. 自動テストと手動テストに分ける

すべてを自動化しようとするしない。

種類	向いている確認	例
自動テスト	同じ入力に対して同じ結果を確認するもの	検証、計算、変換、保存、境界値
手動テスト	人が見て判断するもの	見た目、操作感、文言、視認性
一部自動化	操作手順は自動化できるが判断は人が見るもの	UI操作、エラー表示、画面遷移

三点セットで分類し、分類ごとに自動か手動かを決める。

5. 三点セットと自動/手動の対応

どの仕様を、どの方法で確認するかを先に分ける。

仕様	自動テスト	手動テスト
UC仕様書	主要シナリオ、例外フロー	利用者視点の自然さ
UI仕様書	入力制御、画面遷移、エラー発火	見た目、文言、操作感
ビジネスレイヤー仕様書	検証、計算、保存、状態変更	原則少ない
UIなし仕様	事前条件、事後条件、不変条件	原則少ない

ビジネスレイヤーと UI なし機能は、自動テストに寄せやすい。

6. AI のテスト介入ポイント

AI は判断者ではなく、列挙、生成、比較を担当する。

段階	AI が行うこと	人が判断すること
仕様レビュー	テスト観点、漏れ、曖昧さを列挙する	採用する観点
テスト設計	代表例、境界値、異常系を作る	期待値の正しさ
テストコード生成	単体テスト、UI操作テストを作る	採用、修正、削除
実装レビュー	仕様と実装の差分を列挙する	仕様へ戻すか、コード修正か
回帰確認	変更影響と再実行対象を出す	リリース判断

採用判断、業務判断、安全性、責任は人が持つ。

7. UI あり部分のテスト

UI あり部分は、UI仕様書とUC仕様書を軸に確認する。

確認対象	主な確認方法
画面表示	手動確認、スクリーンショット確認
入力	自動テスト、一部手動確認
ボタン操作	UI操作テスト、手動確認
エラー表示	自動発火確認、文言は手動確認
状態変化	UCシナリオテスト

UI テストだけで業務ルールを確認しない。

業務ルールはビジネスレイヤー側で自動テストする。

8. ビジネスレイヤーのテスト

ビジネスレイヤーは、自動テストしやすい構造にする。

対象	テスト内容
検証	必須、範囲、形式、重複
計算	代表例、端数、境界値
保存	成功、失敗、競合、復元
状態変更	遷移条件、禁止状態、例外
エラー	エラー種別、原因、復旧可能性

UI を起動しなくても、代表例と境界値を確認できるようにする。

9. UI なしライブラリーのテスト

UI がない機能は、三点セットだけに寄せない。

仕様化手法	テストに使うもの
ヘッダー定義	関数、型、戻り値、エラー
VDM-SL	事前条件、事後条件、不変条件
代表例	入力と期待結果
境界値	0、空、上限、下限、不正値
性質ベース	常に成り立つ性質

ライブラリーや中核ロジックは、自動テストに寄せやすい。

10. 大規模システムではテスト対象も分割する

システム全体を一括でテスト設計しない。

分割単位	テストの考え方
業務領域	業務ルール、共通例外、代表シナリオ
アプリ	画面群、権限、利用者フロー
画面	入力、表示、操作、エラー
UC	成功条件、代替条件、例外条件
ビジネス処理	検証、計算、保存、状態変更
UIなしライブラリー	事前条件、事後条件、不変条件
バッチ・外部連携	入出力、失敗、再試行、整合性

分割単位ごとに、仕様、実装、テストを対応づける。

11. 分割単位ごとのテストセット

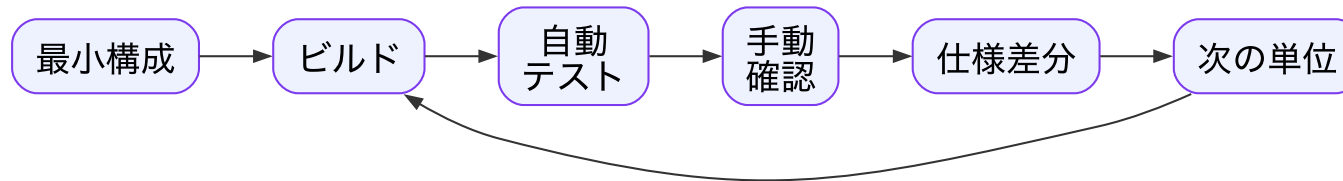
テストも作業資産として対応関係を保つ。



対応表があると、どの仕様をどのテストで確認しているか追える。

12. 小さく動かしてテストする

最初から全体をテストしない。



失敗した場合は、コードだけでなく仕様やプロンプトへ戻る。

13. 生成後レビューとテスト結果

テスト成功でも、仕様どおりとは限らない。

分類	対応
仕様どおり	対応表に反映済みとして記録する
仕様不足	仕様化前ノートへ戻して反映する
生成ミス	実装プロンプトや生成単位を修正する
仕様外追加	削除するか、正式採用するなら仕様へ戻す
未実装	次の生成単位として管理する
テスト期待値の誤り	テストを直す前に仕様を確認する

テスト結果は、仕様と実装の差分を分類する材料である。

14. やってはいけないこと

テストでも、AI 任せにしない。

やってはいけないこと	理由
AI が作ったテストだけで安心する	期待値も推測の可能性がある
仕様のない期待値をテストに入れる	AI の仮決めが固定される
UI テストだけで業務ルールを確認する	ロジックの抜けを見落とす
境界値を省く	失敗しやすい条件が残る
テスト失敗をコードだけで直す	仕様、プロンプト、テストのずれが残る

止める、分ける、仕様へ戻す。

15. この方法で良くなること

三点セットで分類すると、テスト全体を把握しやすい。

効果	内容
テスト全体を把握しやすい	UC、UI、ロジックで棚分けできる
漏れ、重複、偏りを見つけやすい	どの仕様を確認しているか分かる
自動化すべきものが分かる	ロジック、境界値、回帰を自動化できる
手動確認すべきものが分かる	見た目、操作感、文言を人が確認できる
仕様とコードの対応を追える	仕様、実装、テストを対応表で管理できる

AI のテスト生成力を、仕様に結びつけた形で使える。

16. まとめ

テストも分割する。

三点セットで分類する
UCテスト、UIテスト、ロジックテスト

自動と手動に分ける
自動：検証、計算、変換、境界値
手動：見た目、操作感、文言、視認性

AI を支援に使う
列挙、生成、比較

人が採用判断、安全性、責任を持つ。